

# Analizador de Qualidade de Energia

---

Documentação Completa de Arquitetura e Simulação

Projeto de Monitoramento Elétrico (PRODIST)  
Integração Proteus, Wokwi, MATLAB e Python

# 1. Visão Geral do Projeto

Este documento detalha a arquitetura, as metodologias de simulação e as decisões de engenharia adotadas no desenvolvimento do **Analisador de Qualidade de Energia** baseado em ESP32. O objetivo do sistema é monitorar uma rede elétrica AC de 220V, identificar distúrbios em tempo real (seguindo normativas como o PRODIST) e fornecer uma interface rica para análise e tomada de decisão.

O maior desafio do projeto foi a validação do firmware e do hardware através de simulações. Como o microcontrolador ESP32 não possui suporte oficial no simulador Proteus, e o simulador Wokwi não possui suporte avançado para circuitos analógicos complexos (como a rede AC de 220V e transformadores), **a solução de engenharia adotada foi a segmentação da simulação em três domínios integrados:**

| Domínio                         | Ferramenta       | Responsabilidade                                                                                                |
|---------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------|
| 1. Condicionamento Analógico    | Proteus          | Projetar e validar os circuitos físicos de atenuação da rede AC e a detecção de passagem por zero (Zero-Cross). |
| 2. Processamento Digital (DSP)  | Wokwi + MATLAB   | Gerar os sinais matemáticos idênticos à rede, injetar no ESP32 virtual e calcular FFT, RMS e anomalias.         |
| 3. Sistema Supervisório (SCADA) | Python (NiceGUI) | Recepção serial, plotagem de gráficos, Sistema Especialista (Apoio à Decisão) e Datalogger (CSV/PDF).           |

## 2. Etapa Analógica: Condicionamento de Sinal (Proteus)

Para que o ESP32 possa ler a rede elétrica, o sinal de 220V AC ( **$\pm 311V$**  de pico) precisa ser rigorosamente tratado. Durante a fase de projeto, constatou-se que as bibliotecas de módulos comerciais disponíveis para o Proteus não apresentavam confiabilidade adequada. Portanto, optou-se por projetar os circuitos discretos do zero.

## 2.1. Módulo de Leitura de Tensão (Baseado no ZMPT101B)

O sinal da rede passa por um transformador de isolamento e um circuito de condicionamento com amplificadores operacionais. O objetivo deste circuito é duplo:

- **Atenuação e Isolação:** Reduzir a tensão de 220V AC para uma janela segura de operação do microcontrolador.
- **Nível DC (Offset):** O ADC do ESP32 não lê tensões negativas. O circuito adiciona um offset (nível DC contínuo) de aproximadamente **1.65V**, permitindo que a onda senoidal flutue entre **0V** e **3.3V** sem ceifar os semiciclos negativos.

## 2.2. Módulo Zero-Crossing Detector (ZCD)

Para não haver interferência no sinal analógico de leitura, construiu-se um circuito isolado (geralmente acoplado via optoacoplador). Seu comportamento é puramente digital:

- O sinal de saída permanece em **0V** durante todo o ciclo da onda.
- No exato milissegundo em que a onda senoidal cruza o eixo **X (0V)**, o circuito dispara um pulso rápido de **3.3V**.
- Este pulso é fundamental para que o ESP32 calcule o período exato da rede, derivando a **Frequência** e detectando **Saltos de Fase** através de interrupções de hardware (ISR).

### Validação do Proteus

Embora estes circuitos não tenham sido exportados para o Wokwi, a simulação no Proteus com ferramentas de osciloscópio virtual provou matematicamente e visualmente que o hardware projetado fornece sinais de entrada perfeitos e seguros para os limites do ESP32.

## 3. Etapa Digital: Firmware e Matemática (Wokwi + MATLAB)

Uma vez provado que os circuitos entregam uma onda senoidal perfeita de 0 a 3.3V, precisávamos inserir esse sinal no código do ESP32 rodando no Wokwi. Como o Wokwi não possui a rede AC, usamos modelagem matemática.

### 3.1. A Geração do Sinal (MATLAB)

Um script em MATLAB foi criado para calcular os 128 pontos exatos de um ciclo da rede senoidal amostrada a **7680 Hz**. A grande vantagem metodológica desta escolha é a capacidade de injetar distúrbios controlados:

- **Sinal Limpo:** Uma senoide perfeita traduzida para os 12 bits do ADC do ESP32 (valores de 0 a 4095).
- **THD Alto:** O MATLAB soma matematicamente harmônicas de 3ª, 5ª e 7ª ordem à onda fundamental, gerando as matrizes de distorção para testar o algoritmo de Fourier (FFT) do microcontrolador.
- **Ruído (EMI):** O MATLAB injeta ruído branco de alta frequência, criando matrizes para testar o isolamento de espectro de alta frequência.

### 3.2. O Firmware do ESP32 (Wokwi)

No Wokwi, o sinal de Zero-Cross foi facilmente simulado conectando um "Clock Generator" digital a um pino de interrupção, imitando o hardware do Proteus. O firmware realiza as seguintes rotinas de Processamento Digital de Sinais (DSP):

- **Root Mean Square (RMS):** Acumula o quadrado das amostras para calcular o  $V_{rms}$  exato a cada ciclo, identificando afundamentos (*Sag*) e elevações (*Swell*).
- **Deteção de Transientes:** Monitoramento amostra por amostra para identificar *Spikes* de altíssima tensão.
- **Interrupções (ZCD):** Cronometra o tempo entre os pulsos para calcular a frequência da rede e identificar saltos na fase angular.
- **Transformada Rápida de Fourier (FFT):** Converte o array do domínio do tempo para o domínio da frequência, extraíndo a magnitude das harmônicas ímpares críticas (3ª, 5ª e 7ª) e do ruído (EMI).

## 4. Etapa Supervisória: IHM e Sistema Especialista (Python)

O firmware do ESP32 comunica-se via TCP/IP (simulando Serial no Wokwi) enviando strings estruturadas (ex:  $V_{rms}:220.0$ ,  $FFT:220,12,5\dots$ ). O script Python (usando a biblioteca NiceGUI) recebe esses dados e constrói a Interface Homem-Máquina.

## 4.1. Painel de Controle e Gráficos

A IHM foi projetada para atuar não apenas como um display, mas como um ambiente completo de laboratório:

- **Gráficos em Tempo Real:** Um osciloscópio renderiza a onda temporal, enquanto um gráfico de barras plota o espectro da FFT, permitindo ver as harmônicas subindo e descendo dinamicamente.
- **Painel de Simulação:** O usuário possui botões para comandar o ESP32 a alternar a onda lida (Limpo, THD, Ruído) ou injetar distúrbios instantâneos (Gerar Swell, Gerar Sag, Gerar Spike), facilitando a apresentação do projeto.

## 4.2. Sistema de Apoio à Decisão (Sistema Especialista)

O software agrega valor analítico ao invés de apenas mostrar números. Uma Árvore de Decisão foi programada no Python para ler as flags de erro do ESP32 e gerar **Sugestões de Intervenção Técnica**. Por exemplo:

Se o sistema detecta um salto de fase (ZCD out of sync), a IHM não apenas acende um LED vermelho, mas exibe:

*"CRÍTICO (FASE): Salto de fase ou centelhamento detectado. Risco de arco elétrico. Inspeccionar bornes, disjuntores e conexões imediatamente."*

## 4.3. Datalogger e Geração de Relatórios

Para auditoria de qualidade de energia, o sistema permite a criação de logs locais. Quando acionado, o Python grava cada ciclo elétrico com timestamps de milissegundos em um arquivo .csv. Baseado neste arquivo, foi integrado um módulo de PDF que consolida os dados, gera gráficos estáticos via *matplotlib* e emite um Laudo Técnico (PDF) detalhando o histórico do período monitorado.

## 5. Conclusão

A arquitetura dividida superou as limitações individuais de cada simulador. O Proteus garantiu a viabilidade eletrônica; o Wokwi e o MATLAB garantiram a viabilidade matemática do firmware; e o Python entregou a camada de negócios e operação. O projeto atende integralmente aos requisitos de um Analisador de Qualidade de Energia de nível industrial,

preparado para receber o hardware físico (ZMPT101B) mediante pequenas alterações de calibração no firmware.